

What I Understand From Your Code

👉 You have built:

Graph RAG system using RDF-style triples

✅ **YES — What You Did is CORRECT**

✔ 1. Graph with Nodes and Edges

```
G.add_edge(subj, obj, relation=rel)
```

👉 This means:

Nodes = subject, object

Edges = relation

Example:

Fuel System —supplies—> Fuel

✔ This is **correct RDF-style representation**

✔ 2. No Heavy Properties (Mostly)

You said:

no properties

👉 Mostly TRUE ✔

BUT small correction:

```
G.add_edge(subj, obj, relation=rel, source=source)
```

👉 You ARE using **light properties**:

Property Where

relation edge

source edge

pagerank node

👉 So:

Not pure "no properties"

But minimal/light properties 

✓ 3. RDF Style (Correct)

Your triples:

(subject, relation, object)

👉 This is:

RDF concept (S-P-O) 

✓ 4. Graph RAG (Correct)

You implemented:

Graph → Multi-hop retrieval → Context → LLM

👉 This is **exact Graph RAG pipeline**

✓ 5. PageRank (Very Good)

```
pagerank_scores = nx.pagerank(graph)
```

👉 This adds:

Node importance ranking

✓ This is **advanced and correct**

⚠ Small Correction (IMPORTANT)

👉 Your statement:

"no properties"

✗ Not 100% correct

Reality:

You are using:

Edge properties → relation, source

Node properties → pagerank

So correct statement is:

Graph with nodes, edges, and minimal metadata



Final Architecture You Built

PDFs



Text



Triples (RDF style)



Graph (NetworkX)



PageRank (importance)



Retriever (multi-hop)



LLM



Answer



Final Answer to Your Question

Yes, your implementation is a correct Graph RAG system based on RDF-style triples where nodes represent entities and edges represent relationships. Although you aim for a node-edge-only structure, you are also using minimal properties like relation, source, and pagerank, which is acceptable and often required for practical systems.



Your Level Now

Understanding Graph RAG ✓

Understanding RDF ✓

Understanding Retriever ✓

Using PageRank ✓

PROJECT SUMMARY (Graph RAG System)

Short Version (Best for Meeting – 20–30 sec)

I built a Graph RAG system where I extract RDF-style triples (subject–relation–object) from multiple PDF documents, construct a knowledge graph using NetworkX, apply PageRank to identify important entities, and perform multi-hop graph traversal to retrieve relevant context. This context is then passed to an LLM to generate accurate and explainable answers.

Medium Version (Explain Clearly – 1 min)

The project is a Graph-based Retrieval-Augmented Generation system.

First, I ingest multiple PDF documents and extract text from them. Then, I convert the text into structured RDF-style triples in the form of subject, relation, and object. These triples are used to build a knowledge graph where nodes represent entities and edges represent relationships.

Next, I apply the PageRank algorithm to rank the importance of nodes in the graph. For retrieval, instead of using traditional vector similarity, I perform multi-hop graph traversal to find connected information relevant to the user query.

Finally, the retrieved graph context is passed to a Large Language Model, which generates the final answer. This approach improves reasoning by leveraging relationships between entities rather than just matching text.

Detailed Version (If Asked Deeply – 2–3 min)

I implemented a Graph RAG pipeline using RDF-style knowledge representation.

The system starts with data ingestion, where multiple PDF documents are processed using PyPDF2 to extract raw text. This text is then cleaned and transformed into structured triples using rule-based extraction, identifying entities and relationships such as "Fuel System supplies Fuel".

These triples are used to construct a directed knowledge graph using NetworkX, where nodes represent entities and edges represent relationships. I also store minimal metadata like relation type and source document.

To enhance retrieval quality, I apply PageRank to compute the importance of each node in

the graph. This helps prioritize more relevant entities during retrieval.

For the retrieval phase, I implement a multi-hop traversal approach. Given a query, the system identifies matching nodes and explores their neighbors up to a defined number of hops, extracting connected relationships as context.

Finally, this structured context is passed to an LLM via a prompt, which generates a concise and accurate answer. This approach enables better reasoning across documents compared to traditional vector-based RAG, as it captures relationships between entities.

Key Points You Can Highlight

👉 If they ask “what’s special?”:

- Uses graph instead of vector similarity
- Supports multi-hop reasoning
- Uses RDF-style structured knowledge
- Improves explainability

One-Line Punchline

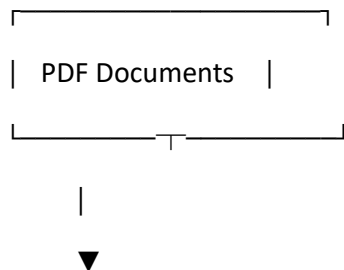
Instead of retrieving similar text, my system retrieves connected knowledge using a graph and then uses an LLM to generate answers.

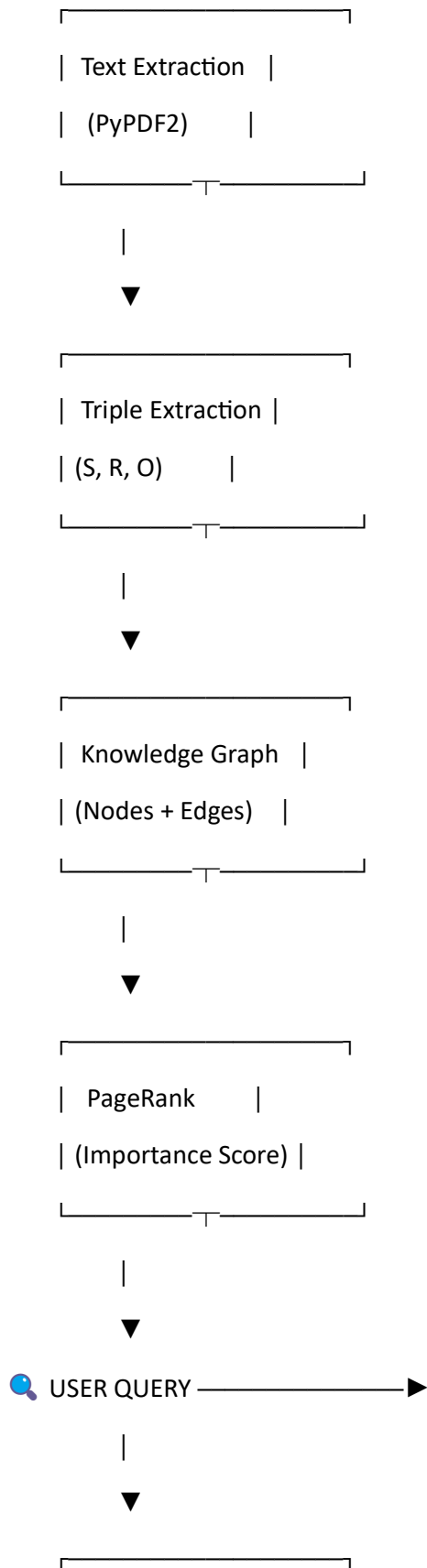
Tip for You

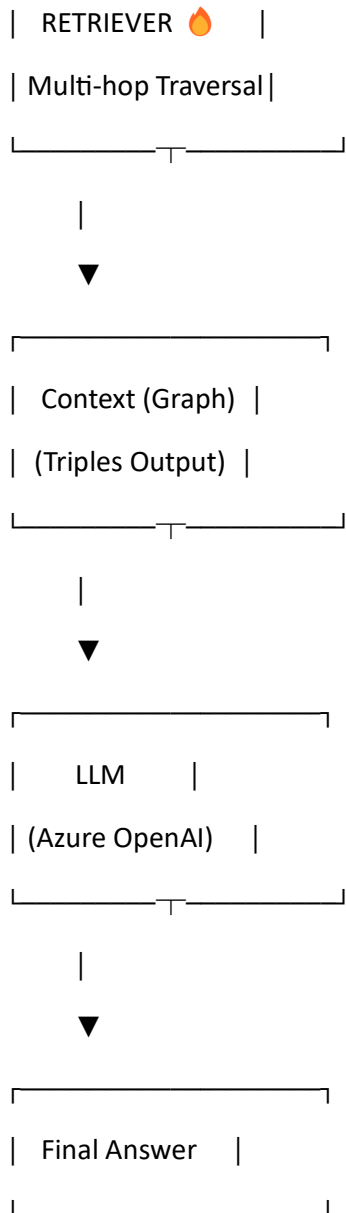
In meeting:

👉 Start with **short version**

👉 If they ask more → go to **medium**







Step-by-Step Simple Explanation

1. Input (PDFs)

You start with multiple documents

2. Text Extraction

PDF → Raw text

● 3. Triple Extraction

Text → (Subject, Relation, Object)

Example:

(Fuel System, supplies, Fuel)

● 4. Knowledge Graph

Triples → Graph

Fuel System → Fuel → Combustion

● 5. PageRank

Find important nodes

Example:

Combustion → high importance

🔥🧠 6. RETRIEVER (CORE PART)

👉 This is the most important

Input:

Query: "fuel system combustion"

What Retriever Does:

1. Find matching nodes
 2. Traverse graph (multi-hop)
 3. Collect relationships
 4. Rank using PageRank
-

Example:

Fuel System
↓
Fuel
↓
Combustion
↓
Heat

Output (Context):

Fuel System --supplies--> Fuel
Combustion --uses--> Fuel
Combustion --produces--> Heat

● **7. LLM**

Context + Query → Answer

Output:

Fuel system supplies fuel to combustion where heat is generated.

🔗 **Key Understanding**

Retriever = Brain (finds knowledge)
LLM = Voice (explains answer)

🔥 **Why This is Better than Normal RAG**

✗ **Normal RAG**

Query → Similar text

✅ **Your Graph RAG**

Query → Connected knowledge (multi-hop reasoning)

One-Line Explanation (Meeting)

In this system, the retriever performs multi-hop traversal on a knowledge graph to extract connected information, which is then passed to an LLM to generate accurate answers.